

Node.js – Övningar Dag 1

Det går också bra att byta ut node till nodemon om man vill i alla övningar.

Övning 1

Titta på exemplet: 2-express-helloworld

1. Skapa en ny mapp: node-dag1-ovn1

2. Kör i terminalen i den nya mappen:

```
npm init
```

```
npm install express
```

3. Skapa en fil som heter app.js som innehåller koden nedan:

```
const express = require('express');  
  
const app = express();  
  
const port = 3000;  
  
app.get('/', (req, res) => {  
  res.send('Hello World!');  
});  
  
app.listen(port, () => {  
  console.log(`Example app listening on port ${port}`);  
});
```

4. Kör node app.js i terminalen och kolla att appen körs och visar texten "Hello World" i webbläsaren när du anropar <http://localhost:3000>

5. Lägg nu till en Request för Post som ska skriva ut t ex "Vi får en Post Request".

(Början på koden):

```
app.post('/', (req, res) => {
```

6. Gå nu till Insomnia (eller Postman) och kolla att texten "Vi får en Post Request" visas när du gör en POST-request till <http://localhost:3000>

Övning 2

Titta på exemplet: 3-express-routing

1. Skapa en ny mapp: node-dag1-ovn2

2. Kör i terminalen i den nya mappen:

```
npm init
```

```
npm install express
```

3. Skapa en fil som heter app.js som innehåller koden nedan:

```
const express = require('express');
```

```
const app = express();
```

```
const port = 3000;
```

```
app.get('/', (req, res) => {
```

```
  res.send('Hello World!');
```

```
});
```

```
app.listen(port, () => console.log(`Example app listening on port ${port}!`));
```

4. Kör node app.js i terminalen och kolla att appen körs och visar texten "Hello World" i webbläsaren när du anropar <http://localhost:3000>

5. Du ska nu göra Routing för en "Att-göra-lista". Det ska inte vara någon funktionalitet utan bara endpoints som skriver ut en text för varje endpoint och tillhörande Request med res.send().

Skapa endpoints t ex app.get('/user' etc för:

todos med Request för get, post, put, delete (som skriver ut en text för varje endpoint och tillhörande Request).

Gör nu också endpoints för kategorier:

categories med Request för get, post, put, delete (som skriver ut en text för varje endpoint och tillhörande Request).

Testa alla endpoints med tillhörande Request i Insomnia eller Postman.

T ex <http://localhost:3000/todos> och <http://localhost:3000/categories>

Övning 3

Titta på exemplet: 4-express-middleware. Men nu ska vi bara använda express-static för att ha en publik mapp med en enkel webbsida dvs ingen routing ännu (som det är i express-middleware-exemplet).

1. Skapa en ny mapp: node-dag1-ovn3

2. Kör i terminalen i den nya katalogen:

```
npm init  
npm install express
```

3. Skapa en fil som heter app.js som innehåller koden nedan:

```
const express = require('express');  
  
const app = express();  
  
const port = 3000;  
  
app.use(express.static('public'));  
  
app.listen(port, () => console.log(`Example app listening on port ${port}!`));
```

4. Skapa en enkel hemsida med en html-fil som kan visa en bild och använda CSS. Alla filer ska ligga i katalogen public som ligger i den nya mappen node-ovn3.

Gör en passande katalog-struktur i public-mappen med t ex img, css

5. Kör node app.js i terminalen och kolla att appen körs och kan visa "hemsidan" från public-mappen i webbläsaren när du anropar <http://localhost:3000>

Övning 4 (fortsättning av övning 3)

1. Lägg till i koden i app.js ovanför app.use(express.static('public'))

Detta för att applikationen ska kunna acceptera json-body samt url (anrop) från ett formulär i html.

```
app.use(express.json());  
  
app.use(express.urlencoded({ extended: true }));
```

4. Skriv en endpoint för Post som heter records

T ex app.post('/records')

Som ska ta emot och skriva ut body genom Insomnia eller Postman. Titta på 4-express-middleware exemplet.

Skriv i JSON-format i Insomnia och välja passande värden att skicka med för en skiva. Tänk på att spara ner de värden som kommer med från req.body. Detta för att inte tillåta vad som helst från klienten. T ex som i kodexemplet: const { firstname, lastname } = req.body;

Gör en validering i koden med vilka värden som inte får vara tomma i req.body. Använd res.status för att skriva ut vad som har blivit fel.

5. Testa din endpoint i Insomnia eller Postman med Post.

6. Skapa en ny html-fil i public-mappen som innehåller ett formulär där man kan skriva in de värden du har valt att din endpoint ska ta emot. Kolla på filen form.html i public-katalogen i

express-middleware hur man gör koden för formuläret med name="" action="" etc.
Formuläret ska gå till <http://localhost:3000/records>
t ex action="<http://localhost:3000/records>"

7. Kör node app.js i terminalen och kolla att appen körs och kan visa "hemsidan" från public-katalogen i webbläsaren när du anropar <http://localhost:3000> samt att endpointen records fungerar när man kör formuläret.

Övning 5

Titta på exemplet: 6-express-rest-books

1. Skapa en ny mapp: node-dag1-ovn5

2. Kör i terminalen i den nya katalogen:

npm init

npm install express

3. Skapa en fil som heter app.js.

4. Gör en egen applikation som ska kunna lägga till och visa filmer (som sparas i en array). På samma sätt som i 6-express-rest-books ska vi använda "middleware" för public genom `app.use(express.static('public'))`

Det ska också finnas endpoints för filmer för att visa (get) och lägga till (post) samt visa en film (på samma sätt som med isbn).

I public (våran frontend) ska det finnas html-kod för att visa alla filmer och lägga till en film i ett formulär (med fetch). I mitt exempel finns det filerna `public/js/fetch-bok.js` för att visa böcker och `public/js/fetch-bok-post.js` för att lägga till en bok. (Du behöver inte kunna anropa endpointen som visar enbart en film med `req.params` i html-koden med fetch utan räcker att testa det i Insomnia eller Postman.)

Det går såklart bra att skriva Javascriptkod i Frontend med Fetch utifrån det ni har gjort i tidigare kurser. Finns tid över så gör gärna Frontend med Vue.

Tänk på för övning 5:

Vilken ordning man bygger upp koden och testar applikationen.

Nu har vi ännu inte någon databas vilket gör att det sedan blir ytterligare ett steg där vi behöver ha bra struktur och upplägg för felsökning. Så bra att få koll på hur man kan lägga upp det redan nu och i vilka steg man gör allting.

Men steg 1 i nuläget är att testa endpoints med tillhörande Request t ex post i Insomnia eller Postman och kolla att det fungerar som tänkt.

Även om det underlättar att använda nodemon istället för node (så vi inte behöver starta om applikationen varje gång vi gör ändringar) så är det en stor del av arbetet med Backend att

göra förändringar i koden och testa i t ex Insomnia. Vi behöver skriva `console.log` i node-koden som visas i terminalen när vi kör applikationen.

Därefter köra html-koden i public i en webbläsare och testa att allting visas och sparas så som det är tänkt när vi kör html-koden. Då har vi också javascript-kod i Frontend att tänka på som har Fetch för att kommunicera med vår Backend. Då behöver vi som vanligt också köra `console.log` för att felsöka vår kod i Frontend.